# RoboCore V2 – ESP32-S3 robot controller (68 x 60 mm, same outline as V1)

Parts follow the SparkFun XRP Controller where possible; circuit drawn independently.

## Power

USB-C in, 1 A Li-ion charger with power path,
ON/OFF switch, 3.3 V buck-boost, 5.5 V boost (VMOT).

File: power.kicad_sch

## MCU

ESP32-S3 module, buttons, RGB LED, buzzer,
LSM6DSOX IMU, I2C pull-ups + Qwiic.

File: mcu.kicad_sch

## Motors

DRV8411A dual H-bridge with current sense,
two XRP motor connectors with encoders.

File: motors.kicad_sch

## IO connectors

Servos, ultrasonic, line follower,
two 14-pin expansion headers.

File: io.kicad_sch

Power flow: USB-C 5 V -> charger -> VSYS -> ON/OFF -> VSYS_SW -> 3.3 V buck-boost and 5.5 V boost (VMOT).

Battery: 1S Li-ion (cells in parallel) on XT30. Programming: USB-C straight to the ESP32-S3.

**RoboCore Zambia / Plastal-Bot Builders**

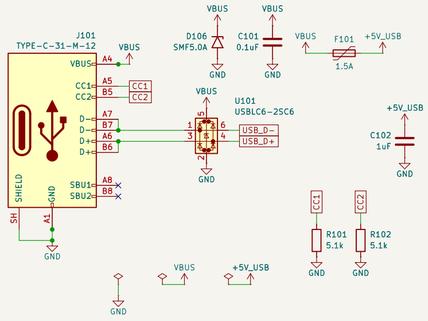
Sheet: /
File: RoboCore_V2.kicad_sch

***Title: RoboCore V2***

| Size: A4 | Date: 2026-10-04 | **Rev: 2.0** |
|---|---|---|
| KiCad E.D.A. 10.0.0 | | Id: 1/5 |

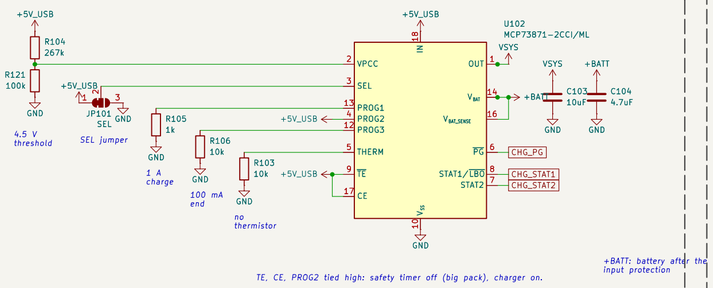
## USB-C INPUT

*USB-C socket for charging and programming. 5.1k on CC1/CC2 asks the source for plain 5 V.*
*USBLC6 clamps static on D+/D−, D106 clamps VBUS surges, and the 1.5 A resettable fuse protects the charger input.*




## BATTERY CHARGER + POWER PATH (MCP73871)

*Charges the 1S Li-ion pack (4.20 V) at up to 1 A and powers the robot (VSYS) at the same time, from USB or the pack.*
*VPCC divider: charging backs off if USB sags below 4.5 V. JP101 SEL: 1–2 = adapter mode 1.65 A (default), 2–3 = USB 500 mA.*

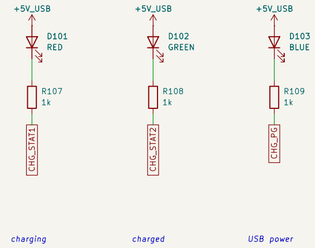



*TE, CE, PROG2 tied high: safety timer off (big pack), charger on.*

*+BATT: battery after the input protection*

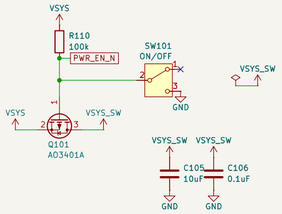
## CHARGE STATUS LEDS

*Driven by the charger: red = charging, green = charge complete, blue = USB power present.*



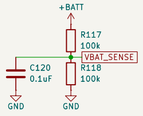

## ON/OFF SWITCH

*Switch ON grounds the P-FET gates: Q101 passes VSYS to VSYS_SW (3.3 V, buzzer, boost enable) and Q102 connects the pack to the boost.*
*Switch OFF: R110 holds both gates high, so everything including VMOT is off (charging still works).*



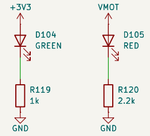

## BATTERY VOLTAGE SENSE

*100k / 100k halves the pack voltage for the ESP32 ADC (IO4).*



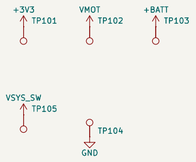

## POWER LEDS

*Green = 3.3 V rail on. Red = motor rail (VMOT) on.*



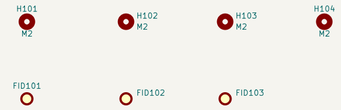

## TEST POINTS

*1 mm pads for bring-up measurements.*



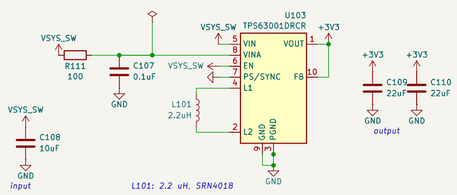

## MECHANICAL

*M2 mounting holes (same places as V1) and fiducials for assembly.*



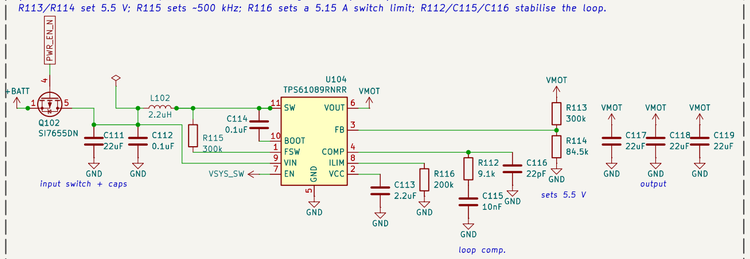

## 3.3 V REGULATOR (TPS63001 BUCK-BOOST)

*Fixed 3.3 V from VSYS_SW (1.8–5.5 V in): holds 3.3 V from a full cell, a flat cell or USB.*
*R111/C107 filter the control supply (VINA). PS/SYNC low = power-save mode at light load.*



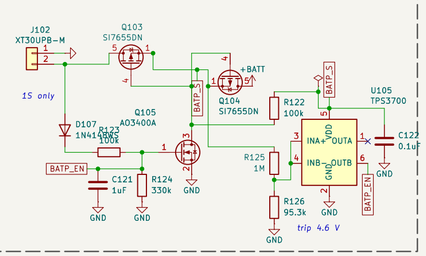

## 5.5 V MOTOR / SERVO SUPPLY (TPS61089 BOOST)

*Boosts the pack (3.0–4.2 V) to VMOT = 5.5 V for the motor driver and servos. Q102 disconnects the pack when the switch is OFF (a disabled TPS61089 still passes VBAT to VMOT through its body diode).*
*R113/R114 set 5.5 V; R115 sets ~500 kHz; R116 sets a 5.15 A switch limit; R112/C115/C116 stabilise the loop.*




## BATTERY INPUT PROTECTION (REVERSE + OVER-VOLTAGE)

*Q103/Q104 back to back: off by default, so a reversed or 2S/3S pack never reaches the board; both ways on when the pack is OK (charging).*
*Q105 switches them on only if XT30+ is positive (D107 blocks a reversed pack; R123/C121 delay ~40 ms so U105 starts first).*
*U105 (TPS3700) pulls the enable low above 4.6 V on the common source: 0.4 V x (1 + 1M / 95.3k). Rated to 4S (16.8 V), reversed to 3S.*





RoboCore Zambia / Plastal-Bot Builders

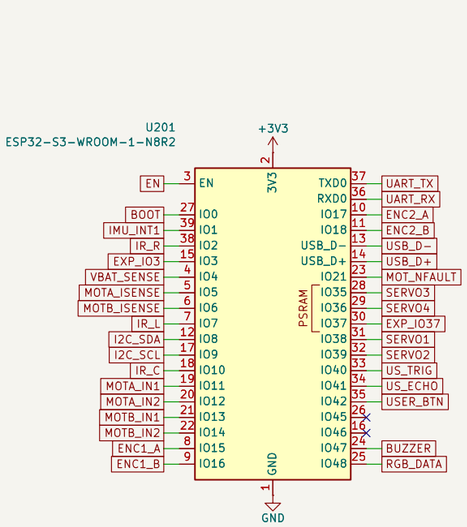

Sheet: /Power/
File: power.kicad_sch

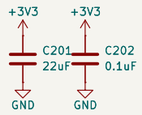
**Title: RoboCore V2 – Power**

Size: A2 | Date: 2026-10-04 | **Rev: 2.0**
KiCad E.D.A. 10.0.0 | Id: 2/5

## ESP32-S3 MODULE

*ESP32-S3-WROOM-1-N8R2: 8 MB flash, 2 MB quad PSRAM. USB D+/D- go straight to the module (upload + debug, no USB-UART chip). IO35-37 are free on N8R2 (quad PSRAM; IO35/36 drive servo 3/4); an R8 (octal PSRAM) module would use them. IO45/IO46 are strapping pins: left open.*



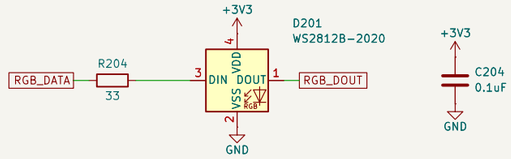




*GPIO MAP*

*IO0 BOOT button*
*IO1 IMU INT1*
*IO2 IR right (ADC)*
*IO3 expansion*
*IO4 battery / 2 (ADC)*
*IO5, IO6 motor 1/2 current*
*IO7 IR left (ADC)*
*IO8, IO9 I2C SDA, SCL*
*IO10 IR centre (ADC)*
*IO11, IO12 motor 1 IN1, IN2*
*IO13, IO14 motor 2 IN1, IN2*
*IO15, IO16 encoder 1 A, B*
*IO17, IO18 encoder 2 A, B*
*IO19, IO20 USB D-, D+*
*IO21 motor nFAULT*
*IO35-37 expansion*
*IO38, IO39 servo 1, 2*
*IO40, IO41 ultrasonic*
*IO42 USER button*
*IO43, IO44 UART TX, RX*
*IO47 buzzer*
*IO48 RGB LED*

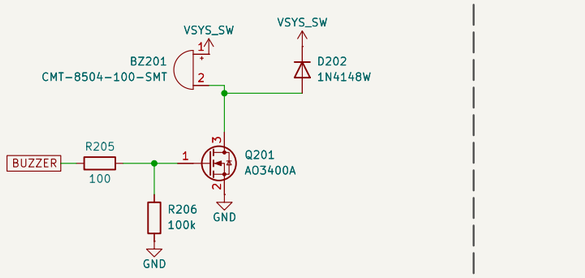
## RGB STATUS LED

*WS2812B-2020-V6 addressable RGB LED on IO48 (33R in series). V6 is rated 3.3-5.3 V (older 2020 versions need 3.7 V). DOUT goes to the expansion header to chain more LEDs.*



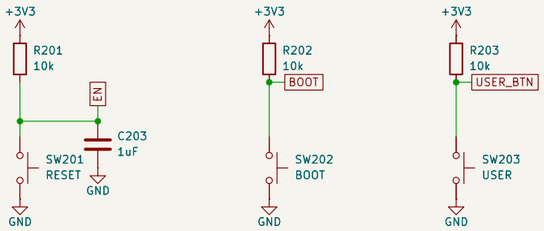

## BUZZER

*Magnetic buzzer on VSYS_SW, switched by a low-side N-FET. A PWM tone on IO47 sets the pitch.*
*D202 absorbs the coil kick-back; R206 keeps the FET off while the ESP32 boots.*




## RESET / BOOT / USER BUTTONS

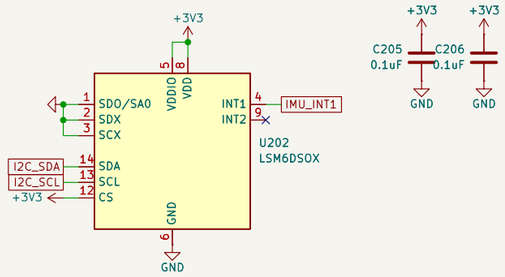
*RESET (EN) restarts the ESP32. Hold BOOT (IO0) and tap RESET to force USB download mode.*
*USER (IO42) is free for your code. Each button pulls its line to GND; 10k pull-ups hold it high.*




*C203: reset delay*

## IMU (LSM6DSOX)

*6-axis accelerometer + gyro on I2C, address 0x6A (SA0 = GND). CS high selects I2C. INT1 -> IO1.*







## I2C BUS + QWIIC

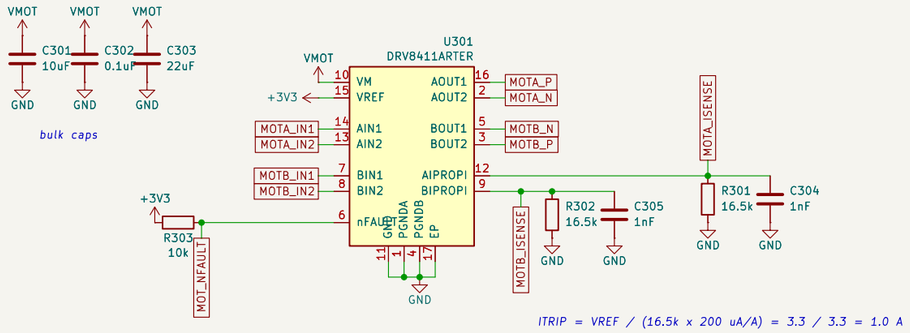
*4.7k pull-ups on SDA (IO8) / SCL (IO9). Qwiic socket (JST SH 4-pin, 3.3 V) for extra I2C sensors.*




RoboCore Zambia / Plastal-Bot Builders
Sheet: /MCU/
File: mcu.kicad_sch
**Title: RoboCore V2 - MCU**
Size: A3 | Date: 2026-10-04 | **Rev: 2.0**
KiCad E.D.A. 10.0.0 | Id: 3/5

## DUAL MOTOR DRIVER (DRV8411A)

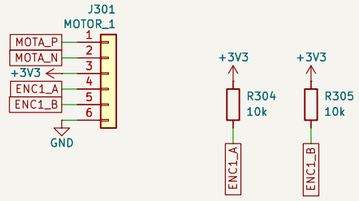
*Two H-bridges powered from VMOT (5.5 V). Per motor, PWM on IN1/IN2 sets speed and the pair sets direction; all low = coast + sleep. IPROPI gives a copy of the motor current (200 uA per A) into 16.5k: the ESP32 reads it, and the driver limits each motor to 1.0 A (JST-SH contact rating).*




*ITRIP = VREF / (16.5k x 200 uA/A) = 3.3 / 3.3 = 1.0 A*

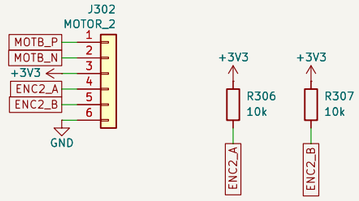
## MOTOR 1 (XRP JST-SH 6-PIN)

*XRP motor plug: 1 M+, 2 M-, 3 3.3 V, 4 encoder A, 5 encoder B, 6 GND.*
*10k pull-ups on the encoder outputs.*




## MOTOR 2 (XRP JST-SH 6-PIN)

*Same plug as motor 1.*
*10k pull-ups on the encoder outputs.*




RoboCore Zambia / Plastal-Bot Builders

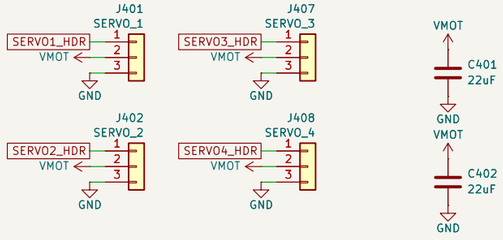
Sheet: /Motors/
File: motors.kicad_sch

**Title: RoboCore V2 – Motors**

| Size: A3 | Date: 2026-10-04 | **Rev: 2.0** |
|---|---|---|
| KiCad E.D.A. 10.0.0 | | Id: 4/5 |

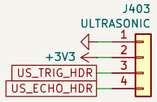
## SERVOS

*Four hobby-servo headers (signal, +, −) as on the XRP. Power is VMOT (5.5 V); C401/C402 are local bulk storage.*
*Servo 1/2 on IO38/IO39, servo 3/4 on IO35/IO36 (all four signals are also on the expansion headers).*



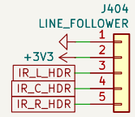

## ULTRASONIC SENSOR

*RoboCore ultrasonic board (CS100A, HC-SR04 timing) on a JST SH 4-pin cable: GND, 3V3, TRIG, ECHO.*
*Same power pin order as the Qwiic port, so plugging it into the wrong 4-pin socket is harmless.*



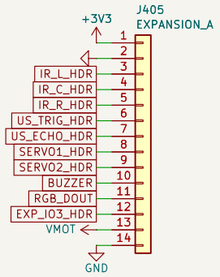

## LINE FOLLOWER (3-CHANNEL IR)

*RoboCore line sensor board (3x TCRT5000) on a JST SH 5-pin cable: GND, 3V3, L, C, R.*
*Analog outputs to ADC1: IO7 left, IO10 centre, IO2 right. 5 pins: it cannot go into a motor (6) or Qwiic (4) socket.*



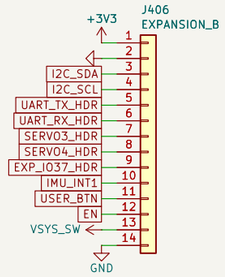

## EXPANSION HEADER A

*Female 2.54 mm header: the sensor, servo, buzzer and LED signals, for prototyping.*



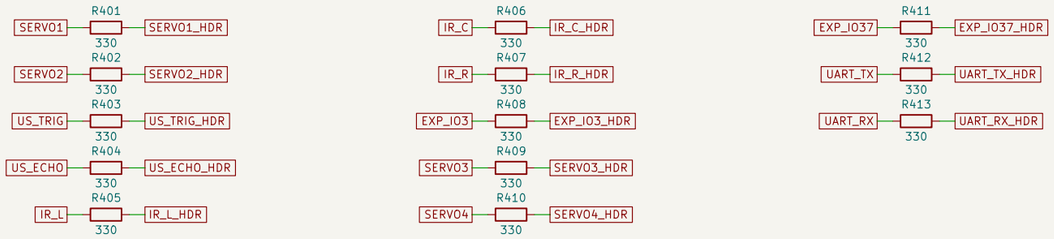

## EXPANSION HEADER B

*Female 2.54 mm header: I2C, UART0, servo 3/4 signals (IO35/IO36), spare IO37, IMU interrupt, USER button, EN and VSYS_SW.*




## HEADER PROTECTION

*330R in series with every ESP32 pin that leaves the board: limits the current if a pin is shorted to a rail or zapped.*
*Nets ending in _HDR are the connector side.*




RoboCore Zambia / Plastal-Bot Builders
Sheet: /IO connectors/
File: io.kicad_sch

**Title: RoboCore V2 – IO connectors**

| Size: A3 | Date: 2026-10-04 | **Rev: 2.0** |
|---|---|---|
| KiCad E.D.A. 10.0.0 | | Id: 5/5 |